

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <termios.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 32
#define MAX_ARGS 32
#define BUF_SIZE 1024

typedef enum { RUNNING, STOPPED } JobState;

typedef struct {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[BUF_SIZE];
    JobState state;
} Job;

Job job_table[MAX_JOBS];
int next_job_id = 1;
pid_t shell_pgid;

// Utility to trim leading/trailing whitespace
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}
```

```

// Add a job entry into the job table
int add_job(pid_t pid, pid_t pgid, const char *command, JobState state) {
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid == 0) {
            job_table[i].job_id = next_job_id++;
            job_table[i].pid = pid;
            job_table[i].pgid = pgid;
            strncpy(job_table[i].command, command, BUF_SIZE - 1);
            job_table[i].state = state;
            return job_table[i].job_id;
        }
    }
    printf("Job table full!\n");
    return -1;
}

// Non-blocking status poll to clear or update process states
void update_job_status() {
    int status;
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid > 0) {
            pid_t result = waitpid(job_table[i].pid, &status, WNOHANG | WUNTRACED | WCONTINUED);
            if (result > 0) {
                if (WIFEXITED(status) || WIFSIGNALED(status)) {
                    printf("[%d] Terminated\t\t%s\n", job_table[i].job_id, job_table[i].command);
                    job_table[i].pid = 0;
                } else if (WIFSTOPPED(status)) {
                    job_table[i].state = STOPPED;
                } else if (WIFCONTINUED(status)) {
                    job_table[i].state = RUNNING;
                }
            }
        }
    }
}

```

```

// Display active jobs list
void list_jobs() {
    update_job_status();
    int found = 0;
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid > 0) {
            found = 1;
            const char *state_str = (job_table[i].state == RUNNING) ? "Running" : "Stopped";
            printf("[%d] %-10s %s\n", job_table[i].job_id, state_str, job_table[i].command);
        }
    }
    if (!found) {
        printf("No active background or stopped jobs.\n");
    }
}

// Resume stopped process in background (bg command)
void resume_in_background(int job_id) {
    int idx = -1;
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid > 0 && job_table[i].job_id == job_id) {
            idx = i;
            break;
        }
    }

    if (idx == -1) {
        printf("bg: job [%d] not found\n", job_id);
        return;
    }

    Job *j = &job_table[idx];

    if (j->state == RUNNING) {
        printf("bg: job [%d] is already running in background\n", job_id);
        return;
    }
}

```

```

// Send SIGCONT to process group to resume execution
if (kill(-j->pgid, SIGCONT) < 0) {
    perror("bg: SIGCONT delivery failed");
    return;
}

j->state = RUNNING;
printf("[%d]+ %s &\n", j->job_id, j->command);
}

// Configure signal delivery, actions, and signal mask
void setup_signals() {
    struct sigaction sa;
    sa.sa_handler = SIG_IGN;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    // Ignore terminal signals in parent shell
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTSTP, &sa, NULL);
    sigaction(SIGTTIN, &sa, NULL);
    sigaction(SIGTTOU, &sa, NULL);

    // Unblock signals in signal mask
    sigset_t block_mask;
    sigemptyset(&block_mask);
    sigaddset(&block_mask, SIGCHLD);
    sigprocmask(SIG_UNBLOCK, &block_mask, NULL);
}

void execute_command(char *line) {
    char original_cmd[BUF_SIZE];
    strncpy(original_cmd, line, BUF_SIZE - 1);

    int is_bg = 0;
    line = trim_space(line);

```

```
size_t len = strlen(line);
if (len > 0 && line[len - 1] == '&') {
    is_bg = 1;
    line[len - 1] = '\0';
    line = trim_space(line);
}

if (strlen(line) == 0) return;

char *args[MAX_ARGS];
int arg_count = 0;
char *token = strtok(line, " \t");
while (token != NULL && arg_count < MAX_ARGS - 1) {
    args[arg_count++] = token;
    token = strtok(NULL, " \t");
}
args[arg_count] = NULL;

if (arg_count == 0) return;

if (strcmp(args[0], "jobs") == 0) {
    list_jobs();
    return;
}

// Built-in 'bg' command
if (strcmp(args[0], "bg") == 0) {
    int target_jid = 1;
    if (arg_count > 1) {
        char *jid_str = args[1];
        if (jid_str[0] == '%') jid_str++;
        target_jid = atoi(jid_str);
    }
    resume_in_background(target_jid);
    return;
}
```

```

pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
    return;
} else if (pid == 0) {
    // Create distinct process group ID for child
    setpgid(0, 0);

    // Restore default terminal signal behavior in child process
    signal(SIGINT, SIG_DFL);
    signal(SIGTSTP, SIG_DFL);

    execvp(args[0], args);
    perror("Command execution failed");
    exit(EXIT_FAILURE);
} else {
    setpgid(pid, pid);

    if (is_bg) {
        int jid = add_job(pid, pid, original_cmd, RUNNING);
        printf("[%d] %d\n", jid, pid);
    } else {
        // Assign foreground terminal control
        tcsetpgrp(STDIN_FILENO, pid);

        int status;
        waitpid(pid, &status, WUNTRACED);

        // Reclaim terminal control back to shell
        tcsetpgrp(STDIN_FILENO, shell_pgid);

        if (WIFSTOPPED(status)) {
            int jid = add_job(pid, pid, original_cmd, STOPPED);
            printf("\n[%d]+ Stopped\t\t%s\n", jid, original_cmd);
        }
    }
}

```

```

    }
}

int main() {
    char input_buf[BUF_SIZE];
    memset(job_table, 0, sizeof(job_table));

    shell_pgid = getpid();
    setpgid(shell_pgid, shell_pgid);
    tcsetpgrp(STDIN_FILENO, shell_pgid);

    setup_signals();

    while (1) {
        update_job_status();

        printf("\nSignalBG Shell > ");
        fflush(stdout);

        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline

        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_command(input_buf);
    }

    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano signal_bg_recovery.c
abhis@MIKHAIL-37:~$ gcc -g -Wall signal_bg_recovery.c -o signal_bg_recovery
abhis@MIKHAIL-37:~$ ^C
abhis@MIKHAIL-37:~$ ./signal_bg_recovery

SignalBG Shell > sleep 60
^Z
[1]+  Stopped                  sleep 60

SignalBG Shell > jobs
[1]  Stopped                  sleep 60

SignalBG Shell > bg 1
[1]+ sleep 60 &

SignalBG Shell > jobs
[1]  Running                  sleep 60

SignalBG Shell > exit
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./signal_bg_recovery
==1031== Memcheck, a memory error detector
==1031== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1031== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1031== Command: ./signal_bg_recovery
==1031==

SignalBG Shell > exit
==1031==
==1031== HEAP SUMMARY:
==1031==    in use at exit: 0 bytes in 0 blocks
==1031==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==1031==
==1031== All heap blocks were freed -- no leaks are possible
==1031==
==1031== For lists of detected and suppressed errors, rerun with: -s
==1031== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```